

Microservices and containers

- **Monolithic and Microservices**
- **There has been a lot of change in the development & deployment since the '80s till now. We can trace the evolution of how organizations have developed, deployed, and managed applications over three distinct phases:**

Development Process

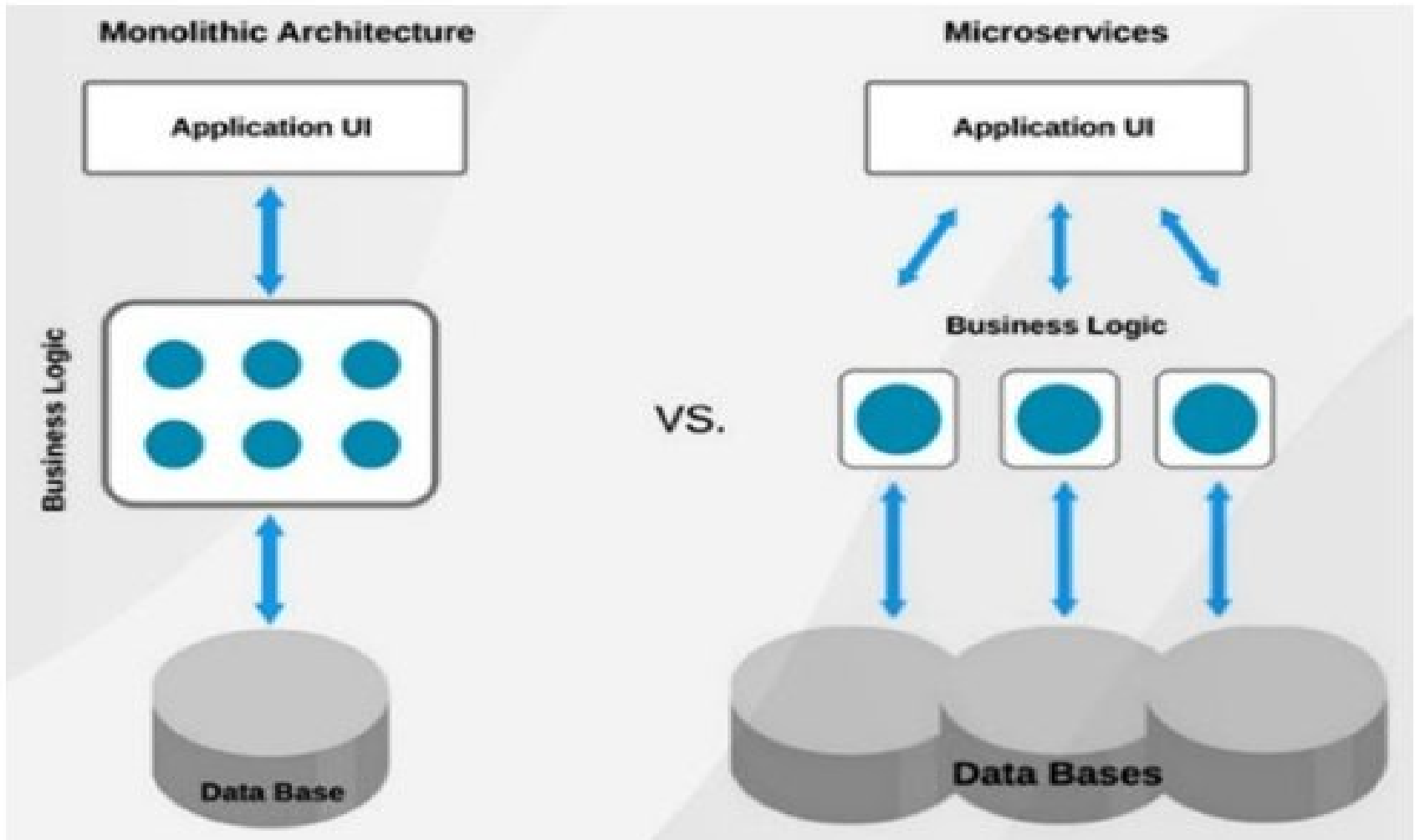
Application Architecture

Deployment and Packaging

Application Infrastructure

~ 1980	Waterfall 	Monolithic 	Physical Server 	Datacenter
~ 1990				
~ 2000	Agile 	N-Tier 	Virtual Servers 	Hosted
~ 2010	DevOps 	Microservices 	Containers 	Cloud
Now				

- **80's – mid 90's: Data Centers with physical servers, running monolithic applications, developed using traditional waterfall methodologies.**
- **Late 90s–00s: Datacenters and hosting providers running Unix/Linux servers, the emergence of virtualization (VMWare, KVM), running n-tier applications, developed using agile methodologies.**
- **~ 2010 – Now: Cloud increasingly replacing data centers and hosting, decomposition of applications into microservices, running on a container infrastructure, managed and orchestrated by Kubernetes, developers, and operations collaborating using DevOps methodologies.**



Monolithic Application

- A monolithic application is constructed as one unit which means it's composed all in one piece. The Monolithic application describes a one-tiered software application within which different components combined into one program from a single platform. The three components are the user interface, the data access layer, and the data store.
- The user interface is the entry point of the application and is also the point of the program that a user will interact with.
- Data access layer is the layer of the program which will wrap the data stored.
- The data store is the most fundamental part of the system and is liable for storing data and retrieving it.

Example of a Monolithic Application



An e-commerce website like early versions of Amazon or Flipkart (before adopting microservices).

- The entire system — user interface, business logic, and database access — was built as one single codebase and deployed together.
- If developers needed to update the “Add to Cart” feature, they had to rebuild and redeploy the whole application.

•

•

•

1. User Interface (UI) Example



- **Example:** The web pages or app screens that customers use to browse products, view prices, and click “Buy Now.”
- This part displays information to users and collects their input.
- Technologies (example): HTML, CSS, JavaScript (front-end built into the same application).

2. Data Access Layer Example



- **Example: A set of functions or modules in the application that handle communication between the UI and the database.**
 - **For instance, a ProductDAO class (Data Access Object) retrieves product details from the database.**
- **Technologies (example): Java JDBC, Python's SQLAlchemy, or .NET Entity Framework within the same codebase.**

3. Data Store Example



- **Example: A relational database like MySQL or PostgreSQL that stores all data — user accounts, product details, and orders.**
- **The application connects directly to this single database to read and write data.**

Advantages of Monolithic:

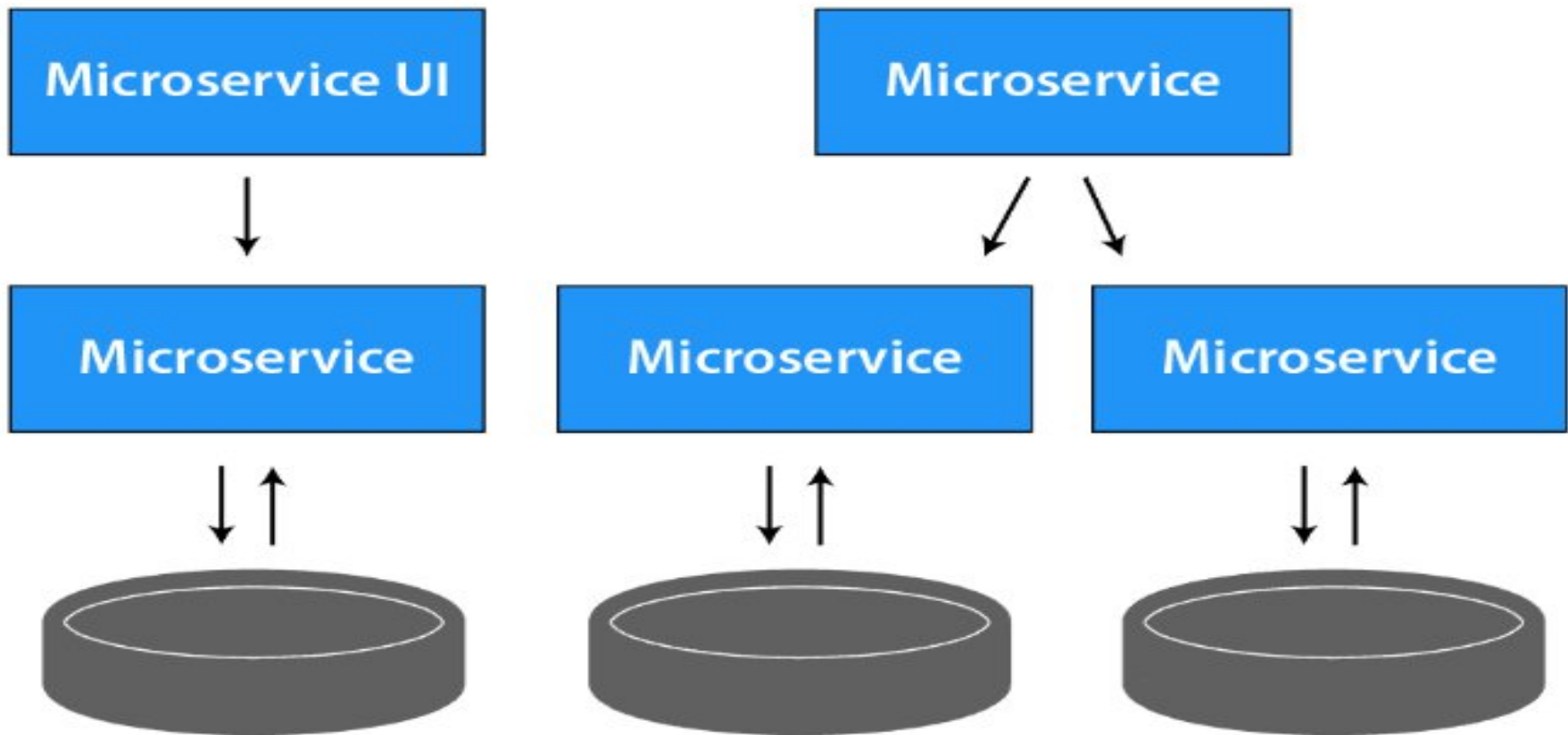


- **Simple to deploy** – In monolithic applications, we don't need to handle many deployments just one file or directory.
- **Easier debugging and testing** – monolithic applications are much easier to debug and test. Since a monolithic app is a single indivisible unit, we can run end-to-end testing much faster.
- **Performance:** Components in a monolith typically share memory which is Quicker than service-to-service communications.

Disadvantages of Monolithic:



- **New technology barriers** – It's extremely problematic to use new technology in a monolithic application because then the whole application must be rewritten.
- **Scalability**- You can't scale components independently, only the whole application.
- **Size** – It becomes overlarge in size with time and becomes difficult to manage.
- **Difficult to understand** – For any new developer joining the project, it's very difficult to grasp the logic of an oversized monolithic application whether or not his responsibility is related to one functionality.



Micro-services Architecture

- a) The idea of microservices is to separate our application into smaller sets and interconnected services rather than building one monolithic application.**
- b) Each module supports a selected business goal and uses a simple and well-defined interface to communicate with other sets of services.**
- c) Each microservice has its own database. Having a database per service is crucial if you wish to learn from microservices because it ensures loose coupling.**

- **Example: *Modern Amazon or Netflix***
 - Instead of one large application, they're made up of many small services, such as:
 - User Management Service
 - Product Catalog Service
 - Payment Service
 - Order Tracking Service
 - Recommendation Service
 - Each service runs independently but communicates with others using APIs.

Example:

- In an online shopping platform, separate services are built for:
 - User Service → handles login, signup, profile.
 - Order Service → manages order placement and history.
 - Payment Service → processes payments and refunds.
- These services communicate using REST APIs or message queues.

Each Module Supports a Business Goal



- **Example:**
 - The Payment Service has one clear goal: handle all payment transactions.
 - The Inventory Service focuses only on updating and managing stock levels.
 - Each module (microservice) does one thing well and can be developed, deployed, and scaled independently.
- **Communication Example:**
 - When a user places an order, the Order Service calls the Payment Service via an API like:
 - POST /payment/process
 - The Payment Service confirms the transaction and notifies the Order Service to proceed.

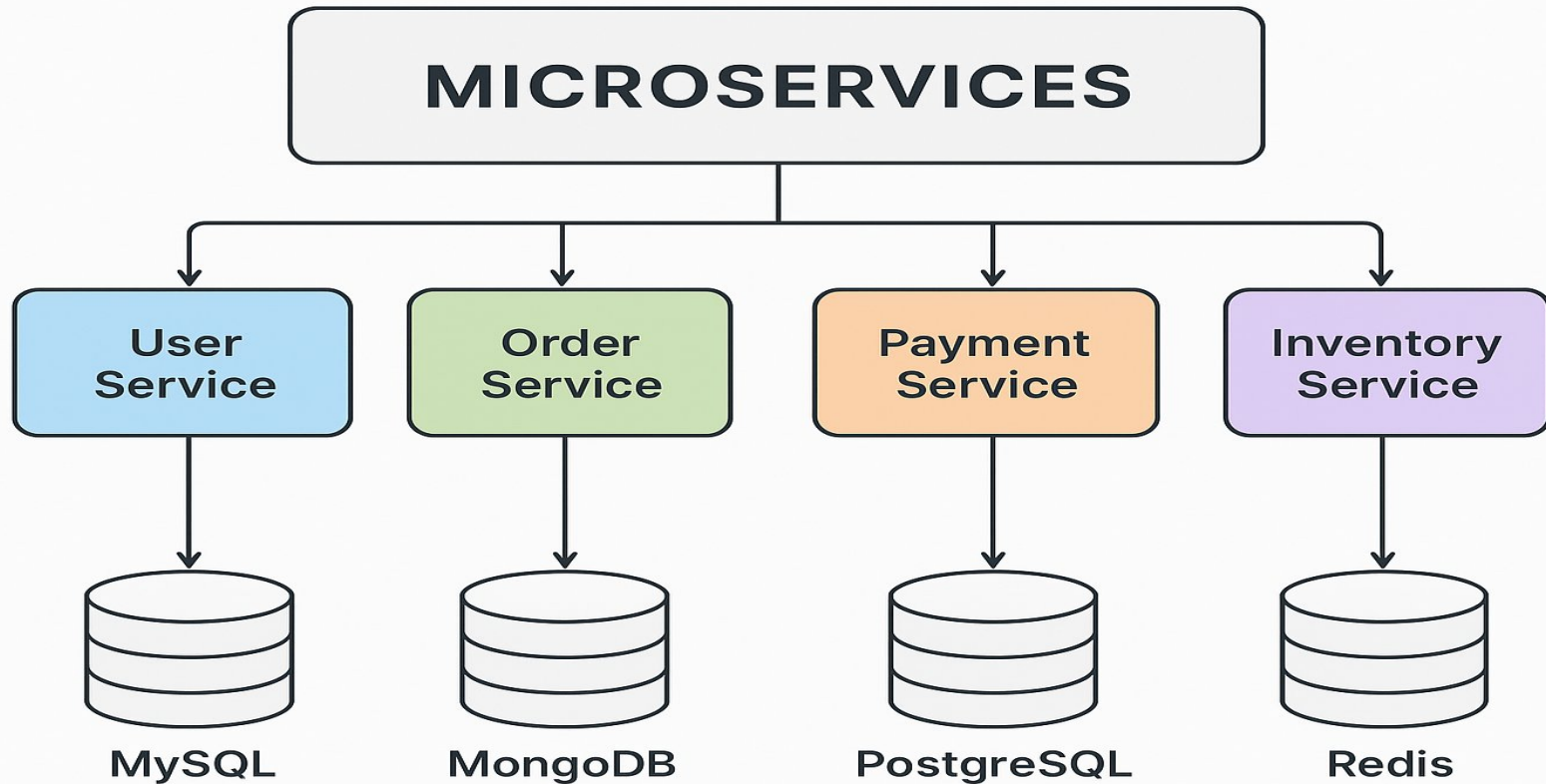
Each Microservice Has Its Own Database



- **Example:**
 - **User Service Database (MySQL):** Stores usernames, emails, passwords.
 - **Order Service Database (MongoDB):** Stores order details, product IDs, delivery status.
 - **Payment Service Database (PostgreSQL):** Stores payment transactions and invoices.
- **This ensures loose coupling — one service can change its database or schema without affecting others.**

Other Example :

Online Food Delivery App (Microservices Example)



Advantages of Microservices:



Advantage	Explanation	Example	Benefit
1. Scalability	Each microservice can be scaled independently based on demand.	During festive sales, only the Order Service is scaled up while others remain constant.	Efficient resource use and better performance.
2. Independent Deployment	Teams can deploy or update one microservice without redeploying the whole system.	Payment Service is updated for new payment gateway without affecting User or Order services.	Faster updates and reduced downtime.
3. Technology Flexibility	Each service can use a different tech stack suitable for its task.	User Service in Python, Order Service in Java, Payment Service in Node.js.	Freedom to choose best-fit tools and technologies.

4. Fault Isolation	A failure in one service does not crash the entire application.	If Recommendation Service fails, other services like Order and Payment still work.	Increased reliability and system uptime.
5. Faster Development & Deployment	Smaller, independent teams can develop, test, and deploy services in parallel.	One team works on Inventory, another on Payment, both releasing updates independently.	Accelerates innovation and delivery speed.
6. Easier Maintenance & Debugging	Each service is smaller and easier to test or fix.	Bugs in User Service can be fixed quickly without touching other modules.	Simplifies debugging and reduces risk.

DISADVANTAGES OF MICROSERVICES ARCHITECTURE



1. Complexity

Managing many services and interactions can be complicated



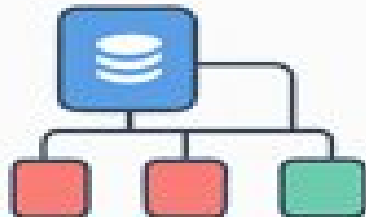
2. Increased Latency

Multiple inter-service calls can add latency to the system



3. Data Consistency

Ensuring data consistency across services can be challenging

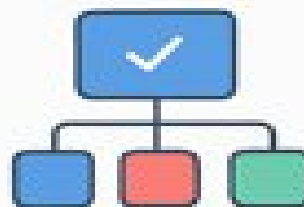


Data Management Complexity

Since each microservice has its own database, maintaining data consistency is hard.

Ex: Updating order status in Order Service must also reflect in Payment Service database

Ex: Risk of inconsistent data of complex transactions

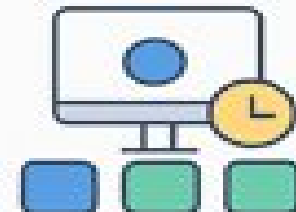


Testing Challenges

Testing across many services is harder than testing a single monolithic application

Ex: End-to-end testing requires running User, Order, and Payment services together

Ex: More complicated integration and regression testing



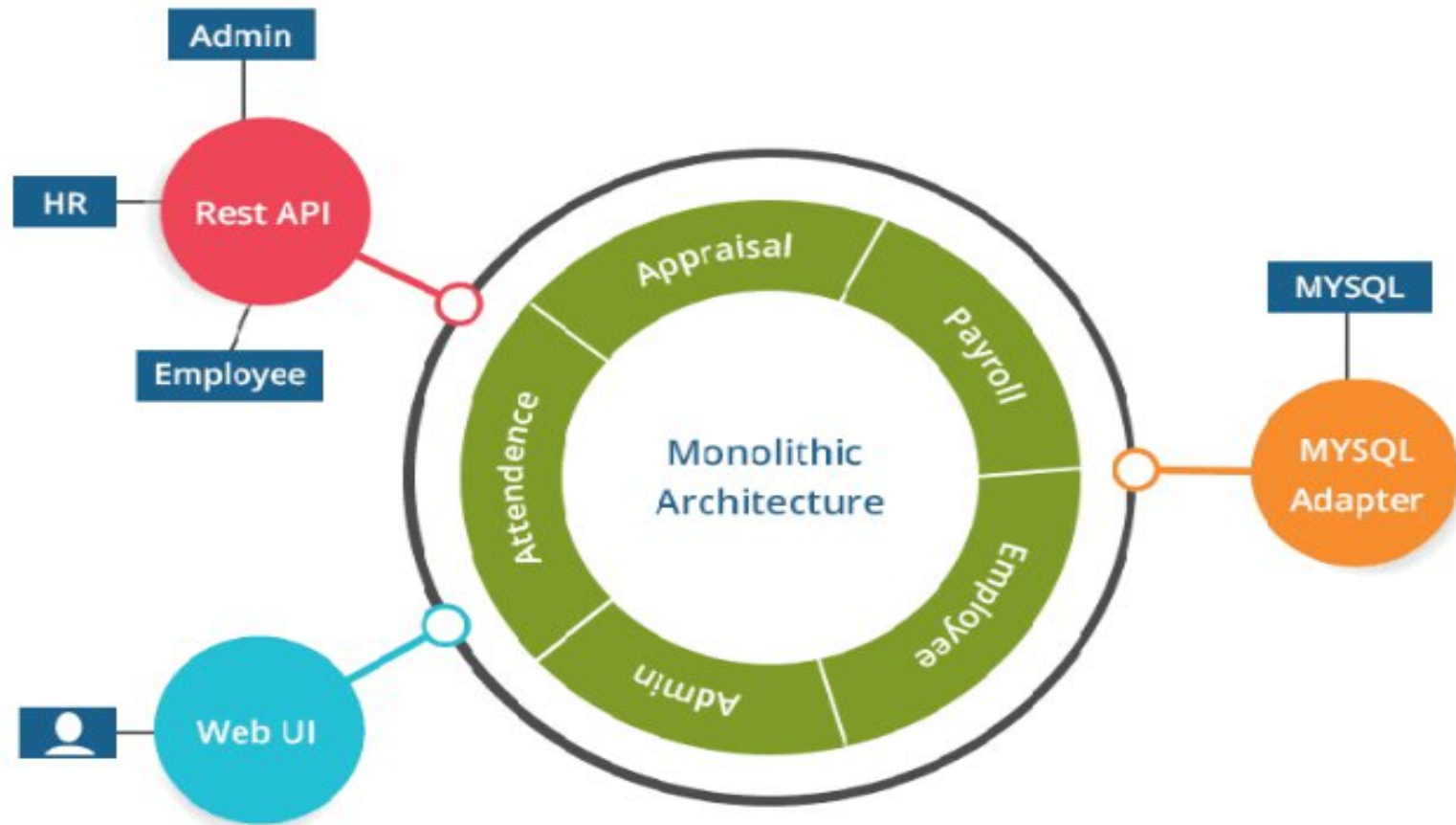
Increased Resource Usage

Each service runs in its own container or process, consuming extra memory and CPU

Ex: 20 microservices running in Docker use more system resources than one monolithic app

Ex: Can increase infrastructure cost

Why MicroServices? ^



- **The good thing about using microservices is that development teams are ready to rapidly build new components of apps to fulfill changing business needs.**
- **In microservices, every application resides in separate containers along with the environment it needs to run.**
- **It allows it to maximize deployment velocity and application reliability.**
- **There's no risk of interfering with the other applications. It also allows optimizing the sources micro services multiply team works on independent services.**

- **In Microservices, we run all the applications on Containers (Docker), so the boot-up time and memory consumption of applications decrease, thus the performance of the application increases.**
- **In organizations, not one or two but multiple numbers of Containers (Docker) are running.**
- **So, to manage them, we use Kubernetes. Kubernetes comes in handy because it provides various facilities like self-healing, scalability, auto-scaling, and declarative state.**

What's the Diff: Monolithic vs Microservices

Monolithic	Microservice
It is built as one large system	It is built as a small independent module
Not easy to scale based on demand	Easy to scale based on demand.
It has a shared database	Each project and module have their own database
Large code base makes IDE slow	Each project is independent and small in size
Continues deployment becomes difficult	Continues deployment is possible here
It is extremely difficult to change technology or language or framework because everything is tightly coupled and depends on each other.	Easy to change technology or framework because every module and project is independent.

Point of Discussion



1. If an online shopping platform on AWS faces high demand for its payment service, how would scaling differ for monolithic vs. microservices architecture?
2. Can you explain what happens during a big sale event—how might microservices help Flipkart manage traffic surges differently than a monolithic system?

Concept of Containers

- Containers are Operating System-level virtualization technologies that allow multiple isolated applications to run on a single host or instance.
- They provide agile, cost efficient, and automated application deployment in which processes are packaged with all the necessary dependencies inside their own separate instances for easy access and portability.
- Containers also offer greater levels of isolation between resources and improve resource utilization compared to traditional virtual machines by using fewer hardware resources.

- Containers are a lightweight alternative to VMs for providing **isolated** operating environments for your workloads.
- They use a different method of **abstracting** resources.
- Instead of using a hypervisor, containers share the **kernel** of the host operating system (OS).
- As a result, they avoid the infrastructure overhead of a full-blown OS and provide only those resources (i.e., **installations**, **dependencies**, and **code**) that your applications actually need.
- This means they can stop and start more quickly in response to **fluctuating scaling** requirements and offer better overall performance than VMs.

Containers

Communication	Use a single network for easier maintenance and configuration of multiple servers
Scalability	Containers can scale easily to adapt to changing workloads, and they can be deployed, replicated, and managed easily. This allows applications to adapt to changing loads with minimal effort.
Management	Container management involves organizing, scaling, and maintaining containerized applications across various environments. This includes container orchestration, which automates the deployment, networking, scaling, and lifecycle management of containers.

Microservices

Modular nature allows individual clusters or services to be on separate networks with their own address spaces
Microservices can scale independently, allowing each component of an app to be scaled depending on the resources it needs. This can efficiently use resources instead of having multiple copies of the entire application.
Microservices performance testing is a critical aspect of developing and maintaining microservices-based applications. This involves assessing performance metrics such as response time, throughput, and the scalability of both individual microservices and the entire application.

Containers and microservices



- Containers and microservices are two important technologies driving the future of development.
- While they share many similarities, there are key differences that must be understood when determining which approach is right for an application.
- Containers enable portability and resource efficiency while offering great scalability options through containerization technology such as Docker or Kubernetes.
- On the other hand, Microservices allow flexibility due to their modular design, allowing applications to scale up without major changes in architecture all the while leveraging communication between services over a network connection; this also makes DevOps easier than ever before!

- To better performance and a lower infrastructure footprint, containerized microservices are typically more robust than a traditional monolithic application.
- For example, if you deploy an entire application to a VM, when the machine goes down, the whole application goes down with it. But, with containers, you can replicate microservices across a cluster of smaller VMs.
- So, in the event of a VM failure, the application will still be able to use the other microservices in the cluster and continue to function.
-

A Simple Container Cluster



Microservices → Many services → Many containers



LOVELY
PROFESSIONAL
UNIVERSITY

Docker Compose helps:

- ✓ Define multiple services
 - ✓ Manage networking
 - ✓ Control dependencies
 - ✓ Scale services
 - ✓ Reproduce architecture easily
- Compose = Practical enabler of microservices (especially in dev/testing)

Docker Compose



Docker Compose is a tool used to define and manage multi-container

Docker applications using a YAML configuration file.

Key idea:

- ✓ **Instead of running containers individually**
- ✓ **Define the entire application stack declaratively**

Docker alone:

- Runs individual containers

Docker Compose:

- Manages groups of related containers

Its like:

- Docker = Container-level management**

Docker Compose = Application-level management

Docker Compose follows:



- ✓ **Declarative configuration**
- ✓ **Infrastructure as Code**
- ✓ **Reproducibility**
- ✓ **Automation**

Need of Docker Compose

Problem Without Compose

Modern applications rarely use a single container.

Typical stack:

- ✓ Web application
 - ✓ Database
 - ✓ Cache
 - ✓ Backend API
 - ✓ Message queue

Running manually:

```
docker run web
docker run db
docker run redis
docker run backend
```

Problems faces

Too many commands

- ✗ Hard to remember parameters
- ✗ Hard to reproduce setup
- ✗ Networking complexity
- ✗ Dependency issues

How docker solve the problem

(A) Multi-Container Management

Define entire system in one file.

- ✓ Centralized configuration
- ✓ Single command execution

(B) Reproducibility

Very strong academic point.

- ✓ Same environment across machines
- ✓ Same configuration for all developers
- “Works on my machine” problem → minimized.

(C) Simplified Networking

Without Compose:

- ✗ Manual network creation**
- ✗ IP management**

With Compose:

- ✓ Automatic network creation**
- ✓ Services communicate via names**

Example:

- Service db reachable at hostname db.**
- Huge conceptual advantage.**

(D) Dependency Management

Control startup order.

- ✓ **Web depends on DB**
- ✓ **Backend depends on Cache**

Compose manages orchestration basics.

(E) Scalability

Scale services easily.

- `docker compose up --scale web=3`
- ✓ No manual container duplication

(F) Cleaner Configuration

Instead of:

- `docker run -d -p 8080:80 --name web ...`

You define:

- ports:
 - "8080:80"
- ✓ Human-readable
- ✓ Version-controlled

3. Basics of Docker Compose



Now move to fundamental building blocks.

(A) Compose File (docker-compose.yml)

Heart of Compose.

- ✓ Written in YAML
- ✓ Defines application stack

Basic structure:

```
version: "3.9"
```

```
services:
```

```
  service_name:  
    configuration
```

•Important teaching note:

•YAML = indentation-sensitive

Spacing matters

(B) Services (Most Important Concept)



Service = Definition / Blueprint of containers

NOT containers directly.

•**Example:**

services:

web:

image: nginx

Conceptually:

- ✓ **Service defines role**
- ✓ **Container = Instance of service**

Analogy:

•**Class → Object**

Service → Container

(C) image vs build

Two ways to specify container source.

Using Prebuilt Image

- **image: nginx**
- **✓ Pull from registry**

Building Custom Image

build: .

- ✓ **Uses Dockerfile**
- ✓ **Builds locally**

Theory distinction:

- ✓ **image → Existing software**
- ✓ **build → Custom software**

(D) ports



Defines host-container communication.

ports:

- "8080:80"

Explain carefully:

✓ HostPort : ContainerPort

Common student confusion → directionality.

(E) volumes



Persistent storage.

- **volumes:**

- **data:/var/lib/mysql**

- **Concept:**

- ✓ **Containers ephemeral**

- ✓ **Volumes persistent**

- **Benefits:**

- ✓ **Data safety**

- ✓ **State preservation**

Compose automatically creates:

- ✓ **Default network**
- Conceptual advantages:**
 - ✓ **Service discovery**
 - ✓ **Built-in DNS**
 - ✓ **Name-based communication**
- Example:**
 - web → connect to db via hostname db**

(G) depends_on

Controls startup order.

- **depends_on:**

- db

- **VERY IMPORTANT theory clarification:**

- ✓ Ensures start order

- ✗ Does NOT ensure readiness

How Docker Compose Works

Compose performs:

- Reads YAML file
- Pulls/builds images
- Creates network(s)
- Creates volumes
- Starts containers
- Connects services

Emphasize:

Compose automates container lifecycle management.

Docker Compose vs Docker CLI



LOVELY
PROFESSIONAL
UNIVERSITY

Docker CLI

Manual commands

Individual containers

Hard to reproduce

Verbose

Docker Compose

Declarative file

Multi-container apps

Easily reproducible

Clean & structured

Basic Commands

Initialization & Setup

- **docker-compose version**
→ **Check installed version**
- **docker-compose config**
→ **Validate YAML file syntax**

Create the file : notepad docker-compose.yml

version: "3.9"

services:

web:

image: nginx

ports:

- "8080:80"



Starting Services

- **docker-compose up**
→ **Start all services (foreground)**
- **docker-compose up -d**
→ **Start in detached mode (background)**
- **docker-compose up --build**
→ **Rebuild images before starting**



Stopping Services

- **docker-compose down**
→ Stop & remove containers, networks
- **docker-compose stop**
→ Stop containers (no removal)
- **docker-compose start**
→ Restart stopped containers

Monitoring & Debugging

- **docker-compose ps**
→ List running services
- **docker-compose logs**
→ View logs
- **docker-compose logs -f**
→ Live logs

- **Executing Commands**
- **docker-compose exec <service> bash**
→ Enter container shell
- **docker-compose up --scale web=3**
→ Run multiple instances

A company wants to deploy a 3-tier web application using Docker Compose:

- **Frontend → React (Node.js)**
- **Backend → Spring Boot API**
- **Database → PostgreSQL**
- **Requirements:**

All services must communicate

- **Environment variables should be used**
- **Data should persist using volumes**
- **Services must start in dependency order**

- **Create docker-compose.yaml file**
- **docker compose up -d**
- **docker compose config --services**

- **docker exec -it postgres-db psql -U postgres- check database**

- **Create a Docker Compose setup for a Node.js application connected to MongoDB.**

Requirements:

- **Node.js app runs on port 3000**
- **MongoDB runs on default port 27017**
- **Use environment variables for DB connection**
- **Ensure MongoDB starts before Node.js**



- **version: '3.8'**
- **services:**
- **app:**
- **image: node:18**
- **container_name: node-app**
- **working_dir: /usr/src/app**
- **volumes:**
- **- ./usr/src/app**
- **command: npm start**
- **ports:**
- **- "3000:3000"**
- **environment:**
- **- MONGO_URL=mongodb://mongo:27017/mydb**
- **depends_on:**
- **- mongo**
- **mongo:**
- **image: mongo:6**
- **container_name: mongo-db**
- **ports:**
- **- "27017:27017"**
- **volumes:**
- **- mongo-data:/data/db**

- **docker compose up -d**
- **docker compose ps**
- **docker compose logs app**
- **docker compose down**

- **Deploy a WordPress website using Docker Compose with MySQL as the database.**
Requirements:
- **WordPress runs on port 8080**
- **MySQL uses persistent storage**
- **Use environment variables for DB configuration**
- **Ensure proper service dependency**



- **version: '3.8'**

- **services:**

- **wordpress:**

- **image:** wordpress:latest

- **container_name:** wordpress-app

- **ports:**

- **- "8080:80"**

- **environment:**

- **WORDPRESS_DB_HOST:** db:3306

- **WORDPRESS_DB_USER:** wpuser

- **WORDPRESS_DB_PASSWORD:** wppass

- **WORDPRESS_DB_NAME:** wpdb

- **depends_on:**

- **- db**

- **db:**

- **image:** mysql:5.7

- **container_name:** mysql-db

- **restart:** always

- **environment:**

- **MYSQL_DATABASE:** wpdb

- **MYSQL_USER:** wpuser

- **MYSQL_PASSWORD:** wppass

MAVEN

BUILD AUTOMATION

Lifecycle • Dependency Management • Plugins • Docker Integration

Why Build Tools Exist

The problems that triggered automation

Manual Compilation Pain

Developers had to remember exact javac commands, classpaths, and dependencies for every build — error-prone and time-consuming.

Dependency Hell

Managing JAR files manually led to version conflicts, missing libraries, and the infamous 'works on my machine' syndrome.

Inconsistent Builds

Without a standard process, builds differed between developers, CI servers, and production — causing unpredictable failures.

Repetitive Tasks

Testing, packaging, and deploying were manual steps that had to be repeated identically every release cycle.

Maven solves all of the above through a standardized, declarative build system.

Project Object Model (POM)

The heart of every Maven project — pom.xml

GroupId: Unique org identifier e.g. com.example

ArtifactId: Project/module name e.g. my-app

Version: Release version e.g. 1.0.0-SNAPSHOT

Packaging: Output format: jar, war, pom

Dependencies: All library declarations live here

Build: Plugin config, source dirs, output

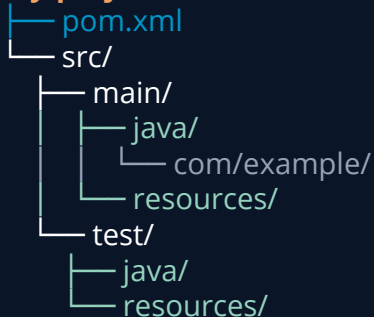
Profiles: Environment-specific configurations

```
<project>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <dependencies>
    <dependency>
      ...
    </dependency>
  </dependencies>
</project>
```

Standard Maven Directory Structure

Convention over configuration

my-project/



target/ ← generated output

src/main/java

Production Java source code. Maven compiles everything here.

src/main/resources

Config files, properties, XMLs bundled into the artifact.

src/test/java

Unit and integration test classes (not bundled in output).

src/test/resources

Test-only config files and fixtures.

target/

Auto-generated build output — classes, JARs, WAR files.

pom.xml

Project descriptor — the single source of truth for the build.

Build Lifecycle Phases

Maven's three built-in lifecycles: default, clean, site

1 validate

Checks project is correct & all required info is available

2 compile

Compiles source code of the project into .class files

3 test

Runs unit tests using a testing framework (e.g., JUnit)

4 package

Bundles compiled code into distributable format (JAR/WAR)

5 verify

Runs integration checks to ensure quality criteria are met

6 install

Installs artifact into local Maven repository (~/.m2)

7 deploy

Copies final artifact to remote repository for sharing

Each phase triggers all previous phases automatically

Parent POM

Centralizing configuration across multi-module projects

What is a Parent POM?

A Parent POM is a special POM that other projects inherit from. Child modules automatically receive all dependency versions, plugin configs, and properties defined in the parent, ensuring DRY (Don't Repeat Yourself) principles across large projects.

Key Benefits:

- Centralized dependency version management
- Shared plugin configuration
- Consistent build behavior
- Single place to update versions
- Spring Boot provides its own parent POM

```
<!-- Parent POM -->  
<parent>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-parent</artifactId>  
  <version>3.2.0</version>  
  <relativePath/>  
</parent>  
  
<!-- Child inherits all configs -->  
<!-- No need to repeat versions -->
```

Dependency Scope

Controlling when and where dependencies are available

Scope	Compile	Test	Runtime	Description
compile	✓	✓	✓	Default scope. Available everywhere — compile, test, and runtime classpaths.
provided	✓	✓	✗	Available at compile/test but NOT bundled. JEE containers (e.g., Tomcat) provide it at runtime.
runtime	✗	✓	✓	Not needed to compile but needed at runtime. JDBC drivers are a typical example.
test	✗	✓	✗	Only for test compilation and execution. JUnit, Mockito are declared test scope.
system	✓	✓	✗	Like provided but references a JAR via explicit path. Discouraged — not portable.
import	—	—	—	Only for <dependencyManagement>. Imports a BOM (Bill of Materials) from another POM.

Transitive Dependencies & Version Conflicts

How Maven resolves the dependency graph

Transitive Dependencies

When you declare $A \rightarrow B$, Maven automatically pulls in all of B's dependencies ($B \rightarrow C$, $C \rightarrow D \dots$). You get the full tree without manually declaring each transitive dep.

Conflict Resolution — Nearest Wins

If two paths lead to the same artifact with different versions, Maven picks the version closest to the project root in the dependency tree (shortest path).

Solutions:

- Explicit direct dependency declaration
- `<exclusions>` to remove unwanted transitives
- `<dependencyManagement>` to enforce a version
- BOM (Bill of Materials) import

Excluding a Transitive Dep:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>lib-a</artifactId>
  <exclusions>
    <exclusion>
      <groupId>com.bad</groupId>
      <artifactId>old-lib</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

```
<!-- Force version via mgmt: -->
<dependencyManagement>
  <dependencies>
    <!-- pin specific version -->
  </dependencies>
</dependencyManagement>
```

Maven Plugins & Execution

Plugins are how Maven does everything

Plugins bind goals to lifecycle phases. A goal is a specific task (like `compile:compile` or `surefire:test`). Each plugin has one or more goals.

Compiler Plugin

Controls Java source/target version for compilation.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <source>17</source>
    <target>17</target>
  </configuration>
</plugin>
```

Surefire Plugin

Runs unit tests. Generates test reports. Skips with `-DskipTests`.

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.1.2</version>
  <configuration>
    <includes>**/*Test.java</includes>
  </configuration>
</plugin>
```

`mvn compile` | `mvn test` | `mvn package` | `mvn verify` | `mvn deploy`

Shade Plugin — Creating an Uber JAR

Packaging all dependencies into a single executable JAR

What is an Uber JAR?

An Uber (German for 'over/super') JAR is a single fat JAR that contains your compiled code PLUS all its dependency JARs merged inside it.

This makes deployment trivially simple — ship one file, run anywhere with `java -jar`.

When to use:

- CLI tools that need to be self-contained
- Microservices deployed to containers
- Spark/Hadoop jobs submitted to clusters
- Anywhere classpath management is painful

⚠ *Watch for class name conflicts from merged JARs*

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.5.0</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
      <configuration>
        <transformers>
          <transformer
            impl="ManifestResourceTransformer">
            <mainClass>com.App</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution></executions>
</plugin>
```

Maven Wrapper (mvnw)

Ensuring consistent Maven version across all environments

The Problem

Different developers and CI servers may have different Maven versions installed, causing 'it works on my machine' build failures.

The Solution

mvnw (Maven Wrapper) is a script committed to your repo. It auto-downloads and uses the exact Maven version specified in `.mvn/wrapper/maven-wrapper.properties`.

How to Generate

Run: `mvn wrapper:wrapper`
This creates `mvnw`, `mvnw.cmd`, and `.mvn/` directory in your project root.

Usage

```
./mvnw clean package
./mvnw spring-boot:run
# Windows: mvnw.cmd clean install
```

✓ No Maven pre-installation required ✓ Version pinned in repo ✓ Works offline (cached in `~/.m2/wrapper`)

Maven & Docker Integration

dockerfile-maven-plugin, Dockerizing apps, Pushing to registries

Integration Overview

- 1. Build JAR**
mvn package produces the application JAR in target/
- 2. Write Dockerfile**
Multi-stage Dockerfile: base image → copy JAR → ENTRYPOINT
- 3. Build Image**
dockerfile-maven-plugin runs docker build as part of mvn install
- 4. Tag Image**
Plugin tags image with project version automatically
- 5. Push to Registry**
mvn deploy pushes image to Docker Hub / ECR / GCR

Dockerfile (multi-stage):

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw package -DskipTests
```

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Build & push via Maven:
mvn dockerfile:build dockerfile:push
```

Pull Maven Image (One-Time)

```
docker pull maven:3.9.10-eclipse-temurin-17
```

```
docker run --rm -it -v ${PWD}:/app -w /app maven:3.9.10-eclipse-temurin-17
```

```
mvn archetype:generate -DgroupId=com.example -DartifactId=demo-app -  
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
mkdir test123
```

```
cd test123
```

```
docker run --rm -it -v %cd%:/app -w /app maven:3.9.10-eclipse-temurin-17 mvn  
archetype:generate -DgroupId=com.example -DartifactId=demo-app -  
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
cd demo-app
```

```
docker run --rm -it -v %cd%:/app -w /app maven:3.9.10-eclipse-temurin-17 mvn clean  
install
```



INT 332 – Devops Virtualization And Configuration Management

Introduction to GitHub Actions



GitHub Actions is a CI/CD (Continuous Integration & Continuous Deployment) platform that allows you to automate workflows directly inside your GitHub repository.

It helps in:

- **Building code**
- **Running tests**
- **Deploying applications**
- **Automating repetitive development tasks**

Analogy:

Think of GitHub Actions like a smart assistant that automatically performs tasks (like testing or deployment) whenever something happens in your repository.



Understanding Workflow Automation



What is a Workflow?

- A workflow is an automated process defined in a YAML file.
- Stored inside `.github/workflows/`
- Contains one or more jobs
- Each job has steps (commands/actions)



Basic Structure of a Workflow

name: My First Workflow

on: push

jobs:

build:

runs-on: ubuntu-latest

steps:

- **name:** Checkout code

uses: actions/checkout@v3

- **name:** Run a script

run: echo "Hello World"

Components of Workflow



- **Event (on) → When to run**
- **Jobs → What to do**
- **Steps → How to do**
- **Actions → Predefined reusable tasks**

Events in GitHub Actions

What are Events?

- **Events are activities that trigger workflows.**

Event	Description
push	When code is pushed
pull_request	When PR is created/updated
workflow_dispatch	Manual trigger
schedule	Runs on cron timing
issues	When issue is created/updated

Triggers in GitHub Actions

What is a Trigger?

A trigger is the condition under which an event runs a workflow.

- Event = "push"
Trigger = "only on main branch"

Example with Filters

on:

push:

branches:

- main

paths:

- - 'src/**'

Types of Triggers

1. Branch-based Trigger

branches: [main, dev]

2. Path-based Trigger

paths: - 'app/**'

3. Tag-based Trigger

tags:- 'v1.*'



4. Time-based Trigger (Cron)

on:

schedule:

- cron: '0 0 * * *'

5. Workflow Directory Structure

repository/

```
|— .github/
|   |— workflows/
|       |— build.yml
|       |— deploy.yml
|
|— src/
|— tests/
|— README.md
```



6. Jobs and Steps

Jobs

A workflow contains multiple jobs

Jobs run in parallel by default

jobs:

build:

test:



Steps

- Each job has steps executed sequentially

steps:

- name: Step 1

run: echo "Hello"

7. Actions (Reusable Components)

What are Actions?

Reusable units of code.

Example:

uses: actions/checkout@v3

8. Runners

What is a Runner?

A server that runs your workflow.

Types:

- **GitHub-hosted (Ubuntu, Windows, macOS)**
- **Self-hosted**

Real-Life Example Workflow



LOVELY
PROFESSIONAL
UNIVERSITY

name: Java CI Pipeline

- **on:**
- **push:**
- **branches: [main]**
- **jobs:**
- **build:**
- **runs-on: ubuntu-latest**
- **steps:**
 - **- uses: actions/checkout@v3**
 - **- name: Set up Java**
 - **uses: actions/setup-java@v3**
 - **with:**
 - **java-version: '17'**
 - **- name: Build with Maven**
 - **run: mvn clean install**



LOVELY
PROFESSIONAL
UNIVERSITY

GitHub Actions Workflows

1. Workflow Triggers in GitHub Actions



- **1.1 Push Trigger**

Runs workflow when code is pushed.

on:

push:

branches: [main]

Use case:

- **Continuous Integration (CI)**
- **Build & test on every commit**

1.2 Pull Request Trigger



- Triggered when a PR is opened, updated, or merged.

on:

pull_request:

branches: [main]

Use case:

- Code review validation
- Testing before merging

1.3 Schedule Trigger (Cron Jobs)

Runs workflow at scheduled intervals.

on:

schedule:

- cron: '0 0 * * *'

Example:

- **Nightly builds**
- **Automated backups**

1.4 Manual Trigger (workflow_dispatch)

Allows user to trigger workflow manually from GitHub UI.

on:

workflow_dispatch:

Use case:

- **On-demand deployments**
- **Testing pipelines manually**

2.Jobs & Matrix Strategies

What is a Job?

A job is a set of steps executed on the same runner.

jobs:

build:

runs-on: ubuntu-latest

Matrix Strategy (Parallel Execution)

- Runs the same job with multiple configurations.

jobs:

test:

runs-on: ubuntu-latest

strategy:

matrix:

node-version: [14, 16, 18]

steps:

- uses: actions/setup-node@v3

with:

node-version: \${matrix.node-version}

3. Steps & Shell Commands

Steps

- Each job consists of sequential steps.

Steps:

- - name: Step 1
- run: echo "Hello World"

Shell Commands



- **Run OS-level commands directly.**

- name: List files

run: ls -la

Quick Quiz



LOVELY
PROFESSIONAL
UNIVERSITY

1. Identify the correct order of execution:

- **A. Steps → Jobs → Workflow**
- B. Workflow → Jobs → Steps**
- C. Jobs → Workflow → Steps**
- D. Steps → Workflow → Jobs**

2. Which of the following is a reusable unit in GitHub Actions?

- A. Job**
- B. Step**
- C. Action**
- D. Runner**



3. A workflow in GitHub Actions is defined using:

- **A. JSON file**
- B. YAML file**
- C. XML file**
- D. Properties file**

4. Where are workflow files stored in a repository?

- **A. /src/workflows/**
- B. /github/actions/**
- C. /.github/workflows/**
- D. /ci-cd/**



5. Which component defines *when* a workflow runs?

- A. Jobs
- B. Steps
- C. Events
- D. Actions



6. Which of the following is a valid push trigger?

A.

- **on: push**

B.

- **on:
 push:
 branches: [main]**

C.

- **trigger: push**

D.

- **event: push**



7. What is the role of a runner?

- A. Stores code**
- B. Executes workflows**
- C. Triggers events**
- D. Builds UI**

8. Jobs in a workflow run:

- A. Sequentially by default**
- B. In parallel by default**
- C. Only manually**
- D. Only on Windows**

9. Steps inside a job run:

- **A. In parallel**
B. Randomly
C. Sequentially
D. Only once

10. Identify the correct order of execution:

- A. Steps → Jobs → Workflow**
- B. Workflow → Jobs → Steps**
- C. Jobs → Workflow → Steps**
- D. Steps → Workflow → Jobs**



11. What does this cron expression mean?

cron: '0 0 * * *'

- A. Every hour**
- B. Every minute**
- C. Every day at midnight**
- D. Every week**



12. You want to test code on multiple Node.js versions simultaneously. What should you use?

- A. Steps**
- B. Matrix strategy**
- C. Runner**
- D. Trigger**



13. If a workflow fails at step 2, what happens?

- A. Step 3 still runs**
- B. Job stops execution**
- C. Workflow restarts**
- D. Next job starts**



14. GitHub-hosted runners support:

- A. Only Linux
- B. Only Windows
- C. Linux, Windows, macOS
- D. None

Practice Questions:



LOVELY
PROFESSIONAL
UNIVERSITY

1. Identify Error

- Find the error in the following YAML:

name: CI Pipeline

on push:

branches:

- main

jobs:

build:

runs-on ubuntu-latest

steps:

- name Checkout code

uses actions/checkout@v3

2.

Workflow Trigger

➤ Write a GitHub Actions workflow that triggers:

On push

Only for develop branch

3. Basic Workflow File



➤ Create a minimal GitHub Actions .yml file that:

Checks out code

Prints "Hello World"

4. Complete CI Workflow

- Write a full .yml file that:
 - Triggers on push
 - Runs on Ubuntu
 - Steps:
 - Checkout code
 - Build project (use echo "Building")
 - Test project (use echo "Testing")

5. Multiple Steps

➤ Write a job with two steps:

- Install dependencies
- Run tests

6.



LOVELY
PROFESSIONAL
UNIVERSITY

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Build

run: ls

7.

- **name: Node CI**
- **on:**
 - **push:**
 - **branches:**
 - **- main**
- **jobs:**
 - **build:**
 - **runs-on: ubuntu-latest**
 - **steps:**
 - **- name: Install dependencies**
 - **run: npm install**
 - **- name: Run tests**
 - **run: npm test**



L OVELY
P ROFESSIONAL
U NIVERSITY

CI /CD

CI/CD is a process of automating code integration, build creation, and testing.

- GitLab CI/CD is a popular tool for implementing CI/CD pipelines.
- It involves steps like code commit, pull request, code merge, build creation, and test case execution.

- Tools like Selenium and JUnit are used for uploading test cases.
- Tools like BlackTurk and SonarQube are used for security vulnerability checks and static code analysis.
- Builds are uploaded to a build repository like SonarTypeNexus and JFrog Artifactory.
- Integration is done using tools like Jenkins, AWS CodePipeline, and GitLab CI.

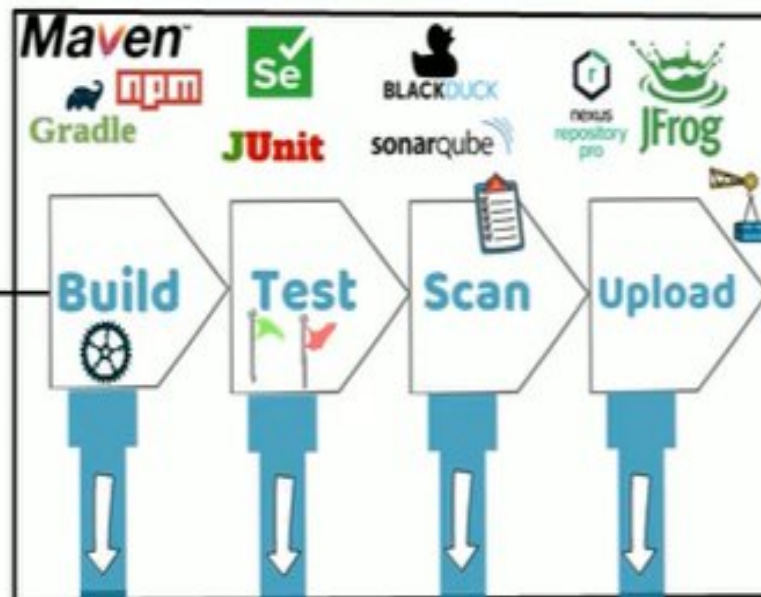
- Deployment is triggered for the dev server first and then for the UAD server after passing sanity checks.
- Fraud deployment is triggered after passing the sanity check on the UAD server.

- Continuous delivery involves continuous feedback and monitoring at all stages of the deployment lifecycle.
- A release cycle includes build, test, scan, upload, development deployment, UAT deployment, and production deployment.

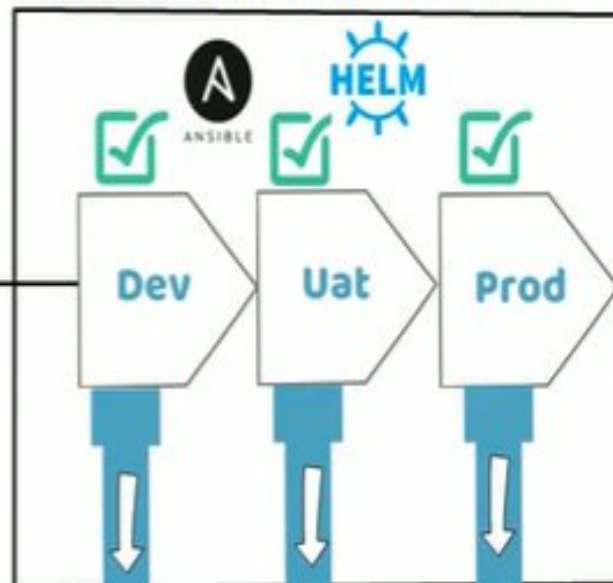


AWS CodePipeline

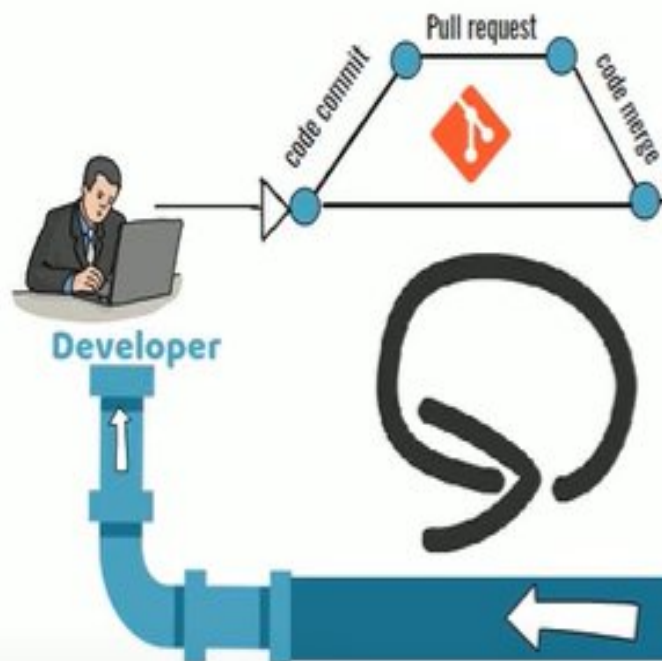
COUNTINUOUS INTEGRATION



COUNTINUOUS DEPLOYMENT/DELIVERY



CONTINUOUS FEEDBACK AND MONITORING



What is Jenkins?

Jenkins is an open-source automation server.

It helps automate the parts of software development related to building, testing, and deploying, facilitating Continuous Integration and Continuous Delivery (CI/CD).

Why Use Jenkins?

- Automates code builds and tests
- Integrates with Git, Docker, Maven, Kubernetes, etc.
- Supports pipelines for end-to-end automation
- Has 1800+ plugins for customization
- Works across different OS platforms

Term	Description
Job/Project	A task to be executed (e.g., build a project)
Build	A single run of a job
Node	A machine where Jenkins runs jobs (Master or Agent/Slave)
Pipeline	A series of steps (scripted or declarative) to define CI/CD workflow
Plugin	Add-on that gives Jenkins extra functionality (e.g., Git plugin, Docker plugin)

Jenkins Setup Basics

1.Install Jenkins

- Download from jenkins.io
- Run with Java (java -jar jenkins.war) or install via Docker

2.Access Jenkins Dashboard

- Go to <http://localhost:8080> in your browser
- Enter the admin password (found in the Jenkins home folder)

Install Suggested Plugins

- **During setup, Jenkins will recommend a few plugins—install them.**

Create First Admin User

- **Set up username, password, and email.**

Creating Your First Job

- Click “New Item”
- Enter a name
- Choose “Freestyle project”
- Configure:
 - Source Code Management (Git)
 - Build Trigger (e.g., Poll SCM or on Push)
 - Build (e.g., Execute Shell or Invoke Maven)
- Click “Save” and then “Build Now”

Think of a Job as:

"Tell Jenkins what to do and when to do it."

Jobs are the core unit of automation in Jenkins.

- Automating Repetitive Work
- Enabling Continuous Integration (CI)
- Setting Up Continuous Delivery (CD)
- Custom Workflows and Pipelines
- Integrations

End-to-End CI/CD Pipeline
Implementation using Jenkins,
GitHub, Maven, and Docker
Add Required Plugins

Pipeline Graph Analysis	Success
Pipeline: REST API	Success
Pipeline: Stage View	Success
Dev Tools Symbols API	Success
Javadoc	Success
JSch dependency	Installing
Maven Integration	Pending
Authentication Tokens API	Pending
Docker Commons	Pending
SSH server	Pending
Git server	Pending
Docker Pipeline	Pending
HTML Publisher	Pending
Role-based Authorization Strategy	Pending
SSH Agent	Pending
Commons Compress API	Pending
Pipeline Utility Steps	Pending
Loading plugin extensions	Pending

```
mkdir neww
cd new
mkdir -p src/main/java/com/example
notepad src/main/java/com/example/Application.java
package com.example;
```

```
import org.springframework.boot.*;
import org.springframework.boot.autoconfigure.*;
import org.springframework.web.bind.annotation.*;
```

```
@SpringBootApplication
@RestController
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

    @GetMapping("/")
    public String home() {
        return "CI/CD Working!";
    }
}
```

notepad Dockerfile

FROM openjdk:17

COPY target/*.jar app.jar

ENTRYPOINT ["java","-jar","/app.jar"]

notepad Jenkinsfile

```
pipeline {  
    agent any
```

```
  
    tools {  
        maven 'Maven'  
    }
```

```
  
    stages {  
        stage('Build') {  
            steps {  
                bat 'mvn clean package'  
            }  
        }  
    }  
}
```

Create pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>app</artifactId>
  <version>1.0</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
```

git init

git add .

git commit -m "added project files"

git branch -M main

git remote add origin <https://github.com/preetkaur18/newww.git>

git push -u origin main

VERIFY ON GITHUB

Jenkinsfile

pom.xml

Dockerfile

src folder

Apply restart with Jenkins

- Go to **Jenkins Dashboard**
- ci-cd-project
- Click on **Configure**
- Scroll down in configuration page
- Find the **Pipeline Section**
- Set **Definition** as:
Pipeline script from SCM
- Set **SCM** as:
Git
- Enter Repository URL:
<https://github.com/preetkaur18/neww.git>
- Enter Branch:
*/main
- Enter Script Path:
Jenkinsfile
- Click **Save**
- **Apply build now**

What is a Jenkins Pipeline?

A pipeline is a set of automated steps that Jenkins follows to build, test, and deploy your application. Think of it as a scripted flowchart for your DevOps process.

Why Use a Pipeline?

- Automate the entire CI/CD process
- Make builds repeatable and version-controlled
- Run complex workflows (parallel stages, conditional logic, etc.)
- Use code to define build steps (called "Pipeline as Code")

Two Types of Pipelines

Type

Description

Declarative

Easier to read and write.
Structured and recommended
for beginners.

Scripted

More flexible but uses Groovy
scripting, best for advanced
users.

What Are Plugins in Jenkins?

Plugins are add-ons that extend Jenkins' core functionality.
They help Jenkins integrate with tools like:

- GitHub, GitLab
- Docker
- Slack
- Maven, Gradle
- JIRA
- Kubernetes
- And much more!

Basically, if Jenkins needs to do anything beyond basic build automation, there's probably a plugin for it.

Why Are Plugins Important?

Without plugins, Jenkins is just a basic automation server. Plugins let you:

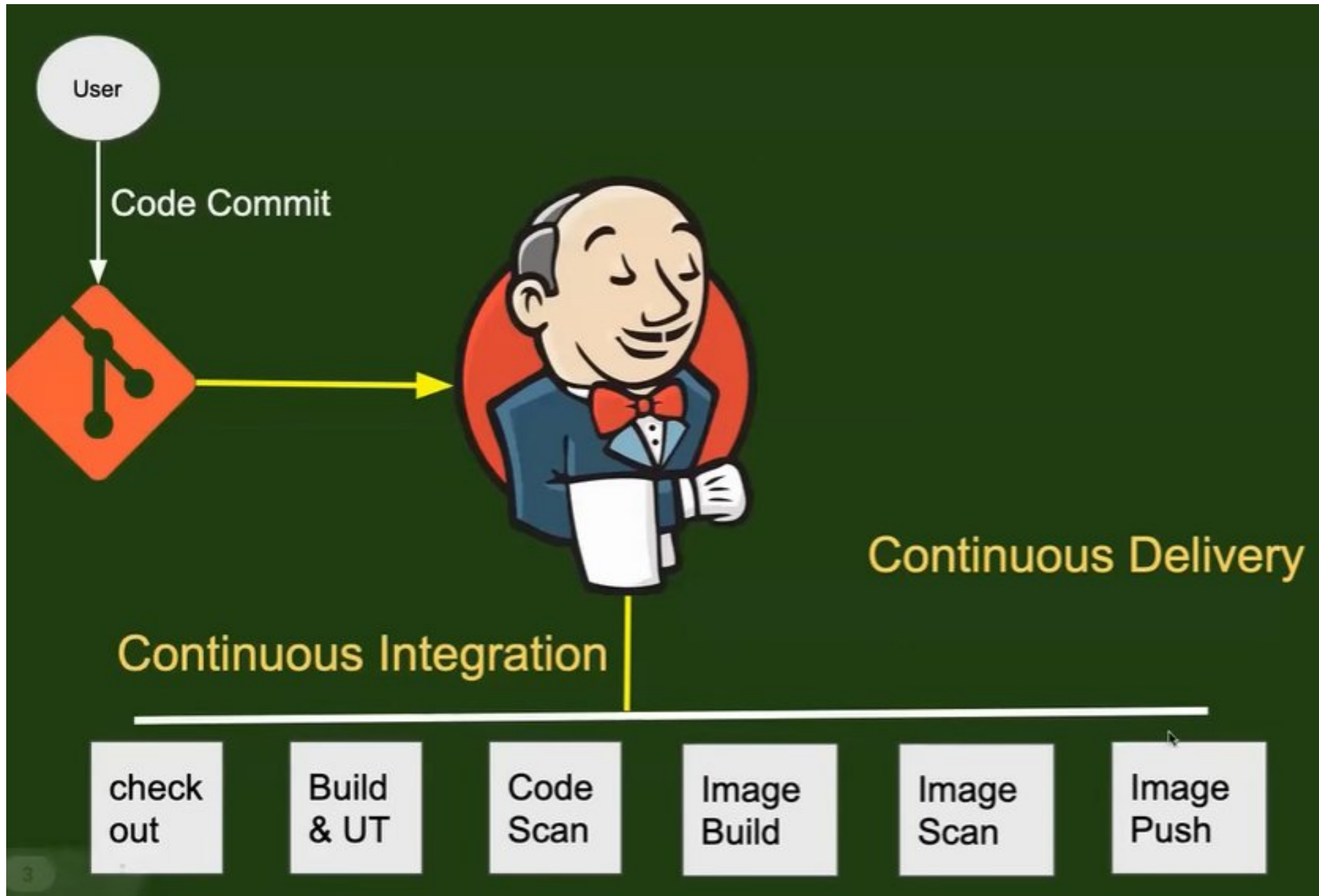
- Connect to source code repositories (e.g., Git plugin)
- Add build tools (e.g., Maven, Gradle plugin)
- Send notifications (e.g., Email, Slack)
- Use containers (e.g., Docker plugin)
- Define complex pipelines (Pipeline plugin)

Plugin Name	What It Does
Git Plugin	Lets Jenkins pull code from Git
Pipeline Plugin	Enables pipeline-as-code using <code>Jenkinsfile</code>
Blue Ocean	A modern UI for Jenkins Pipelines
Docker Pipeline	Lets you run Docker containers inside pipelines
Slack Notification	Sends build status messages to Slack
Email Extension	Sends custom emails on build success/failure
JUnit Plugin	Publishes test results
Credentials Plugin	Securely stores passwords, tokens, and SSH keys

How to Install a Plugin

- Go to Jenkins Dashboard
- Click on Manage Jenkins
- Click on Manage Plugins
- Search in the Available tab
- Click Install without restart or Download now and install after restart

Too many plugins = performance issues or conflicts. Only install what you need.



Overview of Jenkins

- Jenkins is an open-source automation server designed to assist developers in automating the software development lifecycle.
- Initially created for automating the build process for Java, Jenkins now supports a variety of programming languages and technologies.
- By automating repetitive tasks—such as building, testing, and deployment—Jenkins allows developers to concentrate on coding.

Key Features of Jenkins

- Continuous Integration (CI) and Continuous Delivery (CD) are core features, enabling automatic building, testing, and deployment of code changes.
- Jenkins utilizes a rich ecosystem of plugins, which extend its functionality and support automation for diverse tasks in the development workflow.
- Jenkins pipelines allow developers to define the entire build, test, and deployment process as code, facilitating version control and collaboration.

How Jenkins Enhances Workflow

- By automating routine processes, Jenkins reduces the potential for human error and saves time, streamlining the software development process.
- Integrations with other tools in the software development ecosystem make Jenkins adaptable to existing workflows, enhancing productivity.
- Developers can write pipeline scripts using Groovy, a domain-specific language, enabling the definition of complex workflows and task automation.

Importance for Developers and DevOps Engineers

- Jenkins is considered an essential tool in the software development field, providing significant benefits in CI/CD capabilities.
- The automation features of Jenkins enable teams to catch errors and bugs early in the development cycle, improving software quality.
- With its robust automation capabilities and wide-ranging integrations, Jenkins empowers developers to focus on creating high-quality code.

Dashboard ▾

 New Item

 People

 Build History

 Manage Jenkins

 My Views

 Lockable Resources

 New View

Build Queue ^

No builds in the queue.

Build Executor Status ^

1 Idle

2 Idle

[add description](#)

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job →

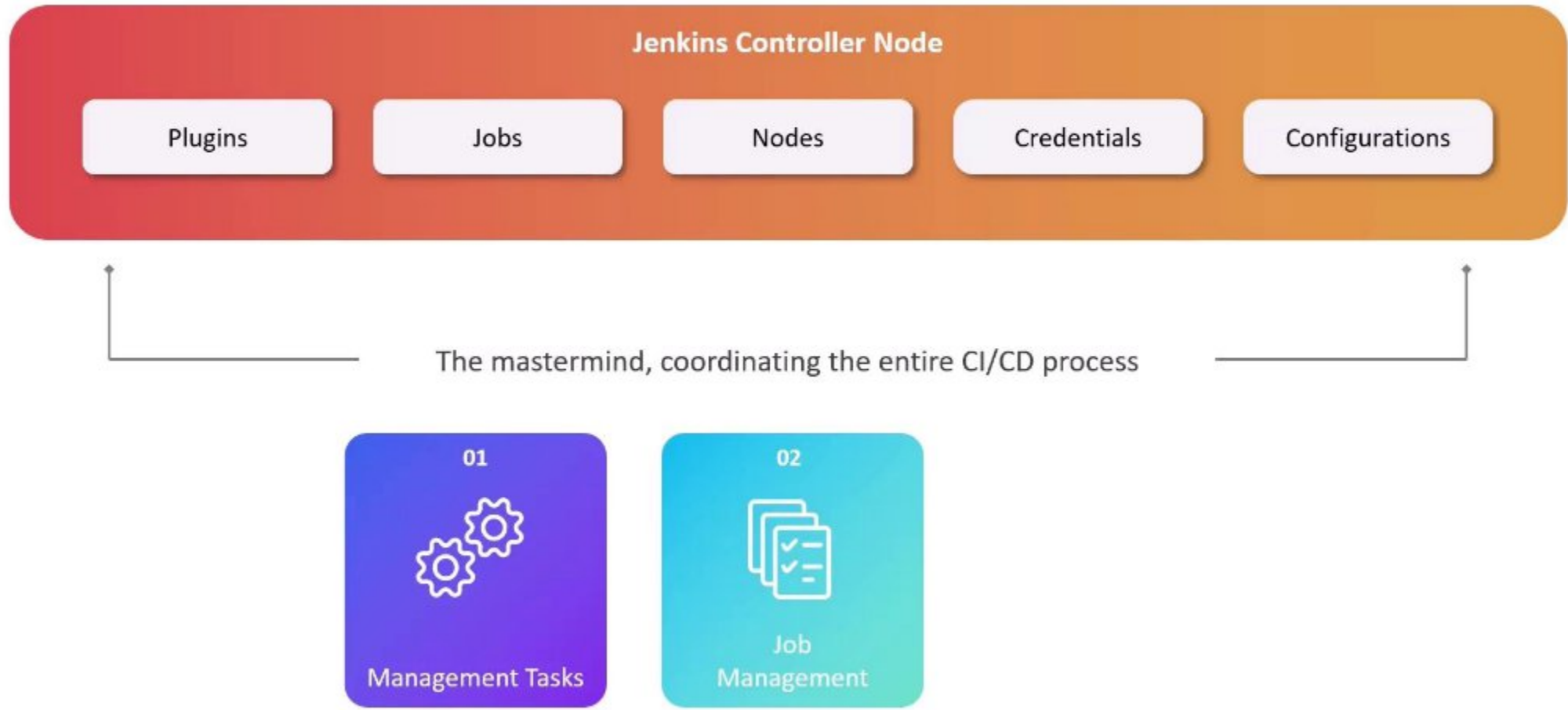
Set up a distributed build

Set up an agent →

Configure a cloud →

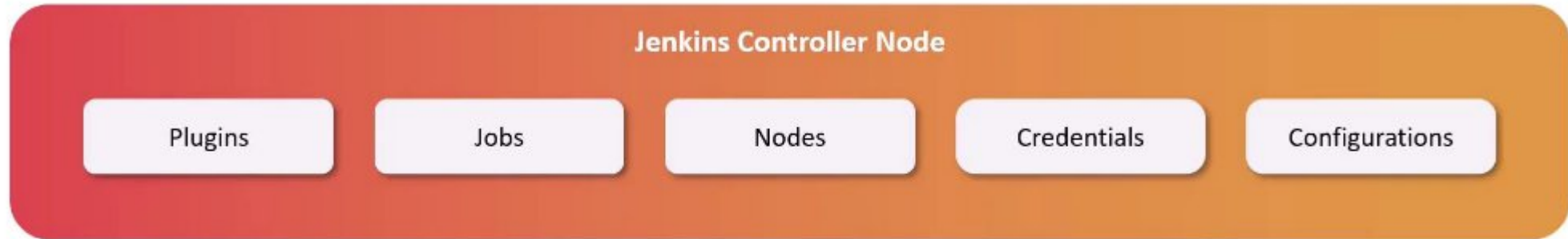
Learn more about distributed builds ↗

Jenkins Architecture





Jenkins Architecture



01

Basic Deployment

Controller and worker node are the same



Limited to
smaller project

02

Advanced Deployment

Separate controller and worker nodes



Prevents
Accidental
Damage



Boosts
Performance



Scalability for
Growth

Jenkins Architecture

